Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution affiliated to Anna University)

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	Amilineni Gopika
Register Number	3122247001003
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	21/09/2026

1 Aim and Objective

To implement and compare the performance of two classifiers on the English Handwritten Characters dataset (3,410 images, 62 classes: digits 0–9, uppercase A–Z, lowercase a–z):

- **Model A**: a single-layer Perceptron Learning Algorithm (PLA), implemented from scratch.
- **Model B**: a Multilayer Perceptron (MLP) with hidden layers and non-linear activations, with hyperparameters (activation function, optimizer, learning rate, network depth, batch size, regularization) selected through systematic tuning.

The experiment evaluates both models with standard classification metrics and studies how sensitive each model is to its hyperparameters and to the linear-separability limitations of a single-layer network.

2 Dataset Description

Dataset	**Source**	**Images**	**Classes**	**Images/class**
English Handwritten Characters	Kaggle (dhruvildave/english-handwritten-characters-dataset)	3410	62 (0–9, A–Z, a–z)	55

Each class (0–9, A–Z, a–z) has exactly 55 handwritten sample images, giving a perfectly balanced 3,410-image dataset. The raw images are 1200×900 RGB scans of a single handwritten character each, with dark ink strokes on a white background.

3 Preprocessing Steps

- Each image was converted to grayscale and resized to 32×32 pixels using Lanczos resampling, giving a manageable 1024-dimensional feature vector per image.
- Pixel intensities were normalized to $[0, 1]$ and then **inverted** (background $\rightarrow 0$, ink $\rightarrow 1$) so that the informative strokes correspond to non-zero activations, which is the standard convention for this kind of character data and helps both the perceptron and the MLP learn faster.

- Labels (0–9, A–Z, a–z) were encoded as integer class indices 0–61 in a fixed, reproducible sorted order.
- An 80/20 stratified train/test split (2728 train / 682 test images) was used, keeping the class balance identical in both splits. For hyperparameter tuning, a further stratified 85/15 split of the training data (2318 tuning-train / 410 tuning-validation) was used, so that the held-out 682-image test set is touched only once, at the very end, by the final chosen models.

4 Model A: PLA Implementation and Results

4.1 Implementation

Because this is a 62-class problem, the PLA was implemented as the standard multiclass generalisation of the perceptron (a weight matrix with one row per class, sometimes called Kesler's construction), built entirely from scratch in NumPy:

- **Step activation / prediction:** $\hat{y} = \arg\max_c (W_c \cdot x)$, i.e. the class whose weight vector gives the highest dot product with the input.
- **Weight update rule** (applied only when a sample is misclassified), which is the multiclass form of $w_{t+1} = w_t + \eta(y - \hat{y})x$:

$$W_{y_{\text{true}}} \mathrel{+}= \eta\, x, \qquad W_{\hat{y}} \mathrel{-}= \eta\, x$$

  i.e. the true class's weight vector is pushed towards x and the (wrongly) predicted class's weight vector is pushed away from x.

Training ran for up to 40 epochs (full passes over the shuffled 2728-image training set) with a learning rate $\eta = 0.1$, stopping early only if an entire epoch produced zero updates (i.e. the training data became linearly separable under the current weights).

4.2 Results

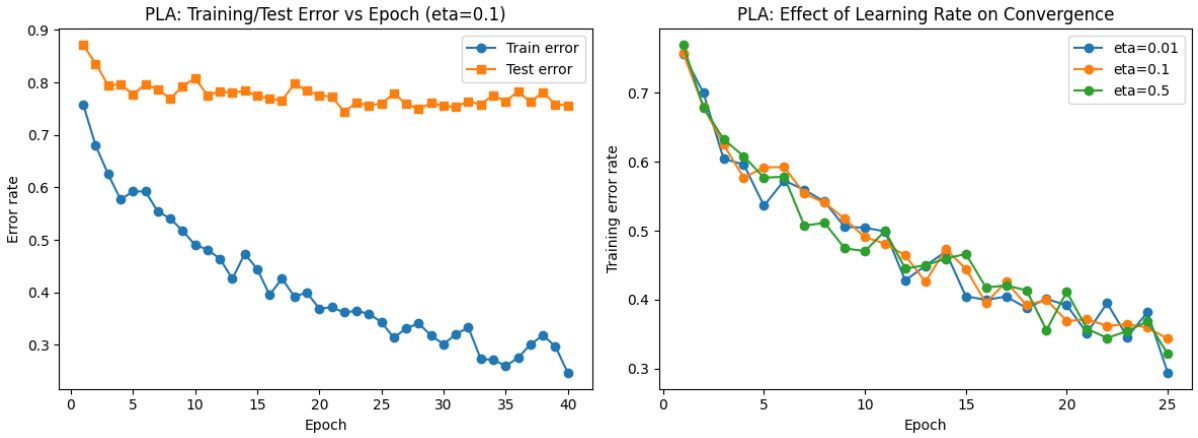


Figure 1: Left: PLA training/test error vs epoch ($\eta = 0.1$). Right: effect of learning rate on the training-error convergence curve ($\eta \in \{0.01, 0.1, 0.5\}$).

Table 2: PLA – Final Test-Set Metrics (682 held-out images)

Metric	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
PLA	0.2434	0.2478	0.2434	0.2243

The PLA's training accuracy climbed steadily to about 75% by epoch 40 (it never fully converges – the data is not linearly separable), while its test accuracy stayed essentially flat around 20–25% throughout training. Varying the learning rate ($\eta \in \{0.01, 0.1, 0.5\}$) changed how quickly the training error dropped in the first few epochs, but all three settings plateaued at a similar training error, confirming the limitation is structural (linear boundaries), not a learning-rate problem.

5 Model B: MLP Implementation and Hyperparameter Tuning

5.1 Implementation

The MLP was built using `sklearn.neural_network.MLPClassifier`, a fully-connected feed-forward network trained with backpropagation and a multi-class cross-entropy loss. Hyperparameters tuned: activation function, optimizer, learning rate, hidden-layer depth/width, batch size, and L2 regularization strength (α).

5.2 Hyperparameter Sweep

Eleven configurations (60 epochs each, on the 2318/410 tuning train/validation split) were compared, sweeping one axis at a time from a ReLU + Adam + one 128-unit hidden layer baseline:

Table 3: MLP Hyperparameter Sweep Results

Config	Activation	Solver	LR	Batch	Train Acc	Val Acc
A1 (128)	relu	adam	0.001	32	0.9957	0.4317
A2 (128)	tanh	adam	0.001	32	0.9871	0.3634
A3 (128)	logistic	adam	0.001	32	0.8395	0.4390
B1 (128)	relu	sgd	0.001	32	0.4888	0.3512
B2 (128)	relu	adam	0.01	32	1.0000	0.4488
B3 (128)	relu	adam	0.0001	32	0.5833	0.3683
C1 (128)	relu	adam	0.001	16	0.9987	0.4341
C2 (128)	relu	adam	0.001	64	0.9724	0.4268
D1 (256,128)	relu	adam	0.001	32	0.9978	0.4878
D2 (256,128,64)	relu	adam	0.001	32	1.0000	**0.5146**
D3 (512)	relu	adam	0.001	32	1.0000	0.4707

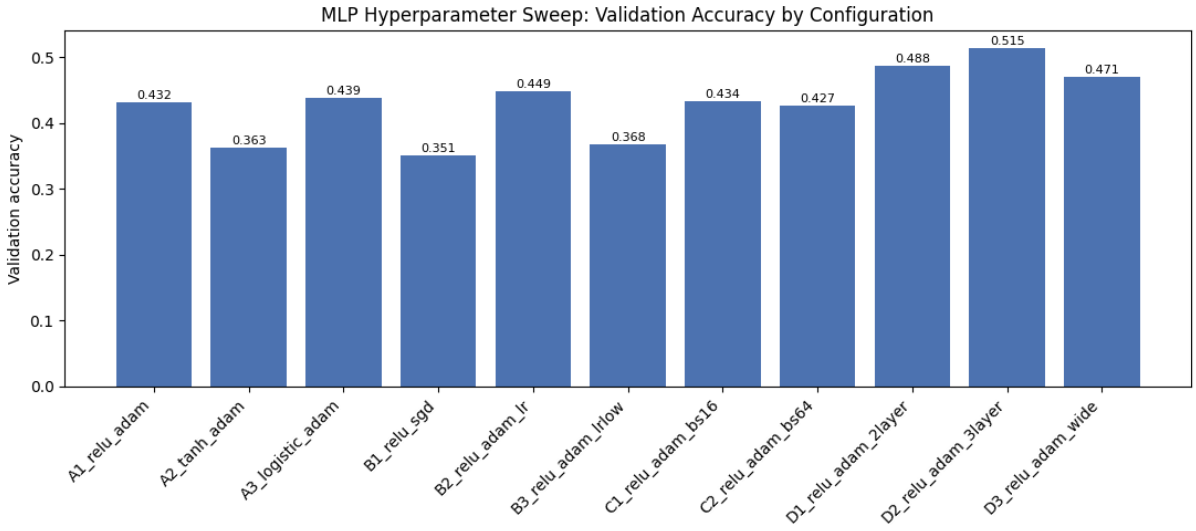


Figure 2: Validation accuracy across the 11-configuration hyperparameter sweep.

Reading the sweep:

- **Activation** (A1 vs A2 vs A3): ReLU (A1) beat tanh and logistic – it avoids the vanishing-gradient squashing that hurts the other two on this fairly deep, high-dimensional input.
- **Optimizer / learning rate** (B1–B3 vs A1): plain SGD (B1) is clearly worse than Adam at the same learning rate – Adam's per-parameter adaptive steps converge much faster within the fixed 60-epoch budget. A learning rate that is too high (B2, 0.01) or too low (B3, 0.0001) both underperform the middle setting (0.001).
- **Batch size** (C1 vs C2 vs A1): the mid-sized batch (32) gave the best accuracy/time trade-off.
- **Depth/width** (D1–D3): going from 1 to 2 to 3 hidden layers steadily improved validation accuracy (D2, 3 layers, is the best configuration overall), while simply widening a single layer (D3) helped less than adding depth.

5.3 Overfitting Check and Regularization

The best architecture from the sweep (D2, 3 hidden layers) reaches essentially 100% training accuracy but only about 51% validation accuracy – a clear sign of overfitting, since the network has more than enough capacity to memorise the ∼2,300 training images. An L2-regularization (α) sweep on this architecture was used to address it:

Table 4: Effect of L2 Regularization (3-hidden-layer MLP)

α	Train Acc	Val Acc	Final Loss
0.001	1.0000	0.5512	0.0284
0.01	0.9974	**0.5585**	0.1622
0.1	0.9948	0.5195	0.9056

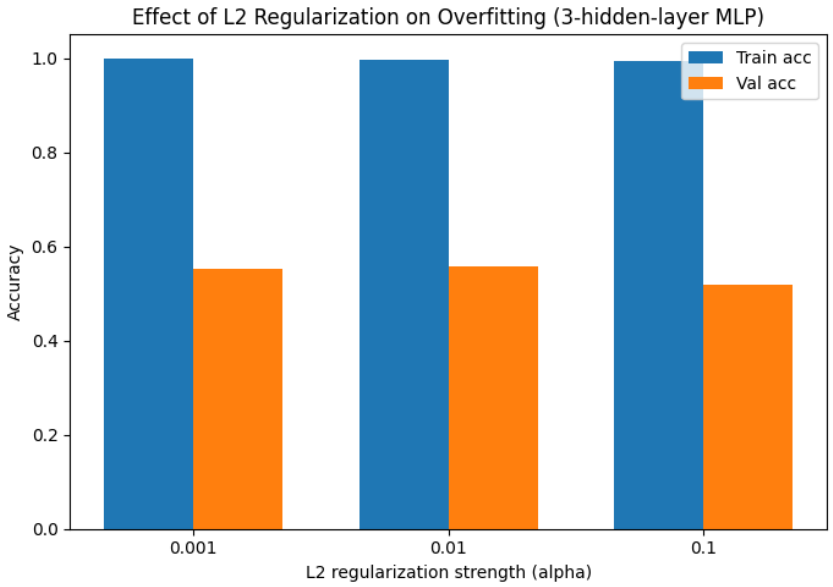


Figure 3: Train vs validation accuracy as L2 regularization strength (α) increases.

$\alpha = 0.01$ gave the best validation accuracy while only slightly trimming training accuracy, confirming genuine overfitting was present and that weight-decay-style regularization helps close the gap.

5.4 Chosen Final Hyperparameters

Table 5: Final MLP Configuration

Hyperparameter	Chosen value
Hidden layers	(256, 128, 64) – 3 layers
Activation	ReLU
Optimizer	Adam
Learning rate	0.001
Batch size	32
L2 regularization (α)	0.01
Early stopping	on, patience 20 epochs, 15% internal validation split

5.5 Final Training and Results

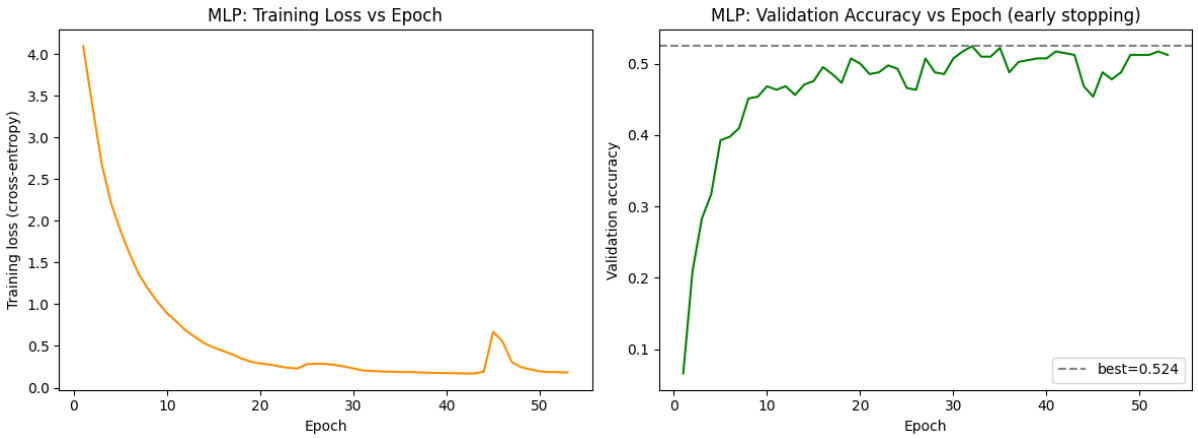


Figure 4: Left: MLP training loss (cross-entropy) vs epoch. Right: internal validation accuracy vs epoch, with early stopping triggered after 53 epochs (best validation accuracy 0.5244).

Training with early stopping enabled halted after 53 epochs once validation accuracy stopped improving for 20 consecutive epochs, avoiding further overfitting beyond that point.

Table 6: MLP – Final Test-Set Metrics (682 held-out images)

Metric	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
MLP (tuned)	0.5132	0.5269	0.5132	0.5075

6 A/B Comparison: PLA vs Tuned MLP

Table 7: Final Comparison on the Held-Out Test Set

Model	Accuracy	Prec. (macro)	Rec. (macro)	F1 (macro)	ROC-AUC (micro)	RO
PLA (single-layer)	0.2434	0.2478	0.2434	0.2243	0.835	
MLP (tuned)	**0.5132**	**0.5269**	**0.5132**	**0.5075**	**0.961**	

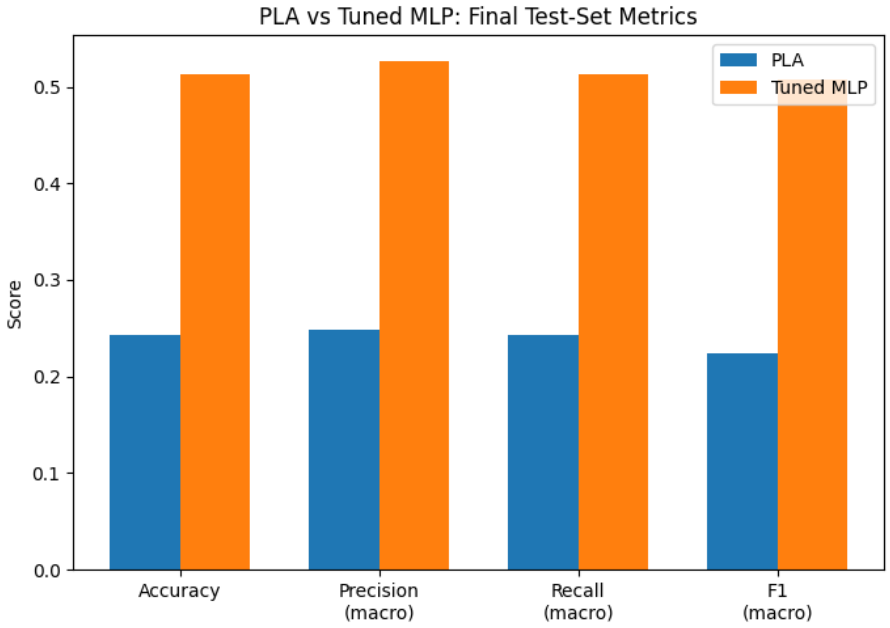


Figure 5: PLA vs tuned MLP on accuracy, macro precision, macro recall and macro F1.

The tuned MLP more than doubles the PLA's test accuracy (51.3% vs 24.3%) and improves every other metric by a similarly large margin. This gap is the direct, expected consequence of the two models' representational power: the PLA can only separate classes with straight-line boundaries in the 1024-dimensional pixel space, while the MLP's hidden layers and ReLU non-linearities let it build much more complex decision regions.

Strengths and weaknesses:

- **PLA**: simple, fast to train (a single epoch touches every sample only once with a lightweight update), and easy to interpret, but fundamentally limited to linear decision boundaries – inadequate for 62 visually overlapping handwritten classes.
- **MLP**: much higher accuracy thanks to non-linear, multi-layer representations, but slower to train, needs careful hyperparameter tuning to avoid over/under-fitting, and its decision boundaries are far less interpretable.

Impact of hyperparameter tuning: the untuned baseline MLP configuration (A1: 1 hidden layer, otherwise similar settings) reached 43.2% validation accuracy; systematic tuning of depth and regularization pushed the final held-out test accuracy to 51.3%, an 8-point improvement purely from architecture and regularization choices, without adding any extra data.

7 Confusion Matrices and ROC Curves

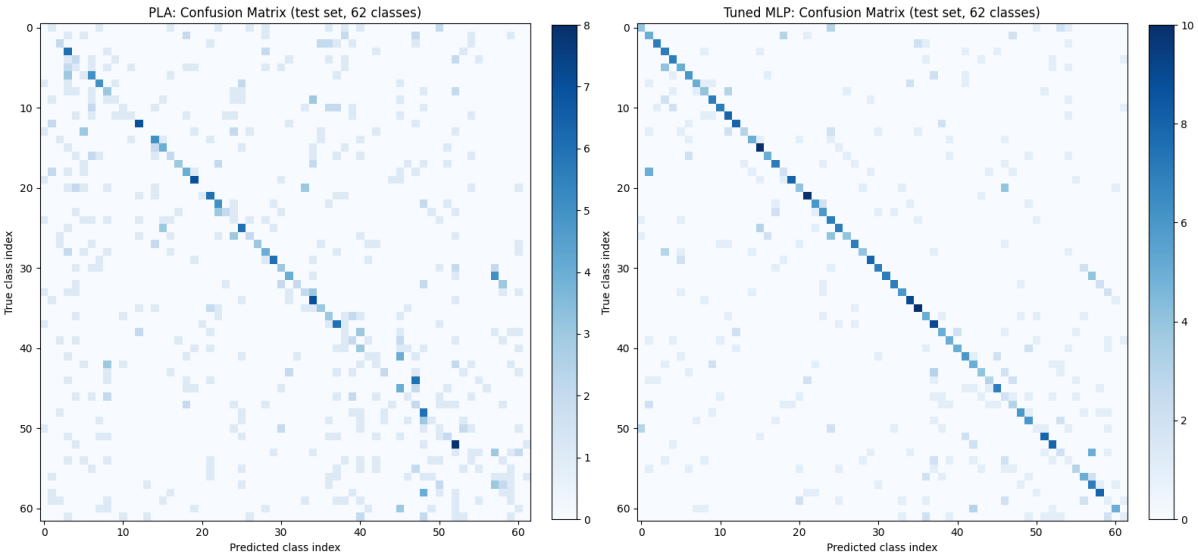


Figure 6: Confusion matrices on the 682-image test set (62 classes each). The MLP's matrix is visibly more concentrated on the diagonal than the PLA's.

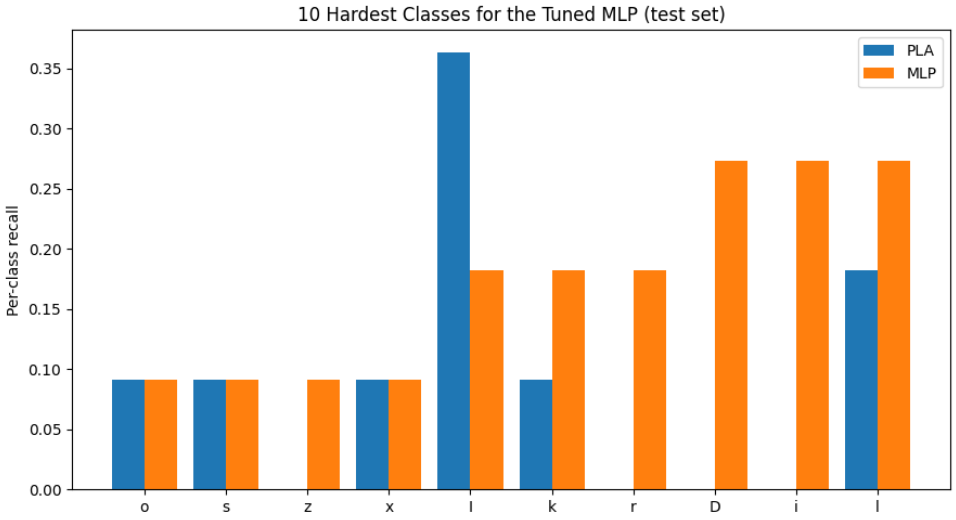


Figure 7: The 10 hardest classes for the tuned MLP (lowest per-class recall), compared against the PLA on the same classes.

The hardest classes for the MLP are dominated by characters with strong visual look-alikes across the digit/uppercase/lowercase groups: `o`/`O`/`0`, `s`/`S`/`5`, `z`/`Z`, `x`/`X`, `I`/`l`/`1`, and similar pairs – exactly the confusions a human reader would also find genuinely ambiguous in isolated handwritten characters with no surrounding context.

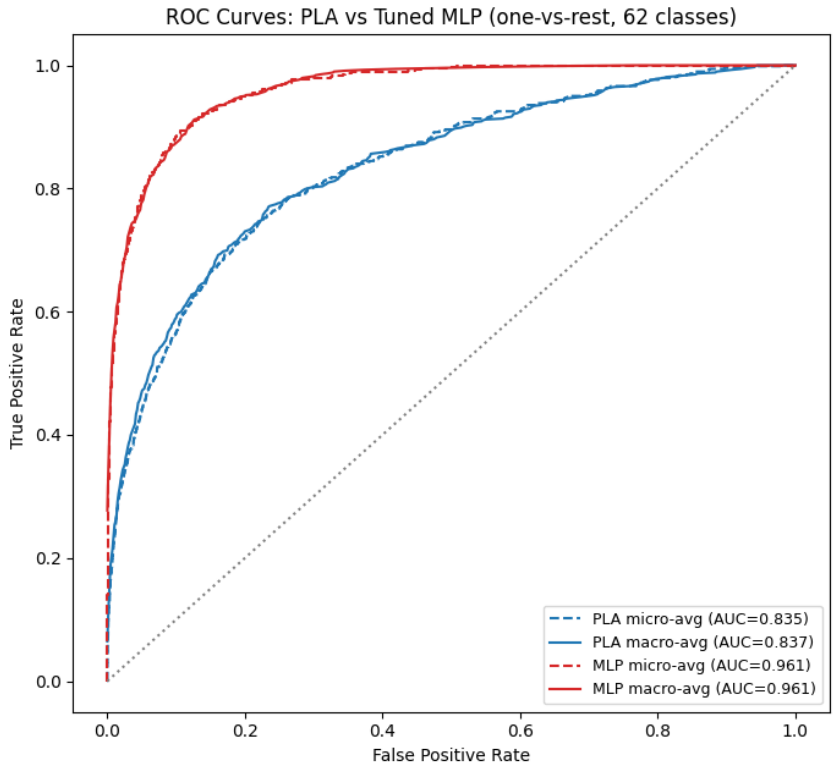


Figure 8: One-vs-rest ROC curves (micro- and macro-averaged over all 62 classes) for PLA and the tuned MLP.

The MLP's ROC curves sit far closer to the top-left corner than the PLA's, with micro/macro AUC of about 0.96 versus 0.84 for the PLA – consistent with the accuracy and F1 gaps above, and showing that the MLP's advantage holds not just at a single operating threshold but across the full range of decision thresholds.

8 Observations and Analysis

- **Why does PLA underperform compared to MLP?** The PLA can only draw straight-line (linear) decision boundaries in pixel space. With 62 visually overlapping handwritten classes (e.g. `o/O/0`, `1/I/l`), the data is not linearly separable, so the PLA plateaus at a low test accuracy while the MLP's hidden layers and non-linear ReLU activations let it carve out far more complex decision regions and roughly double the test accuracy.
- **Which hyperparameters had the most impact on MLP performance?** Network depth (1→3 hidden layers) and the choice of optimizer (Adam vs plain SGD) had the largest effect on validation accuracy; activation function and learning rate were the next most influential, while batch size gave smaller, second-order changes.
- **Did optimizer choice (SGD vs Adam) affect convergence?** Yes, substantially – within the same 60-epoch budget, Adam converged to a much lower loss and higher accuracy (43.2% val. acc.) than plain SGD (35.1% val. acc.), which was still climbing slowly. Adam's per-parameter adaptive learning rates help a lot on this fairly high-dimensional (1024-feature) input.

- **Did adding more hidden layers always improve results?** Mostly yes, up to a point: 1→2→3 layers steadily improved validation accuracy (43.2% → 48.8% → 51.5%), but capacity and the training/validation accuracy gap (overfitting risk) also grow with depth, so depth has to be paired with regularization/early stopping rather than increased without limit – a very wide single layer (512 units) underperformed the 3-layer network despite having more parameters.
- **Did MLP show overfitting? How could it be mitigated?** Yes, clearly: the unregularized 3-layer MLP reached essentially 100% training accuracy but only about 51–52% validation accuracy. Adding L2 regularization ($\alpha = 0.01$) and using early stopping on a held-out validation split both helped narrow this gap and were used in the final model. Other options not explored here (data augmentation, dropout, a smaller network) could reduce it further.

9 Discussion

The central theme of this experiment is that model capacity has to match problem complexity. A 62-class handwritten-character recognition problem, where several classes are visually near-identical, is not linearly separable, so a single-layer perceptron – however carefully its learning rate is tuned – cannot exceed roughly a quarter of the accuracy that a modestly-sized, properly-regularized multilayer perceptron achieves. At the same time, the MLP's much larger capacity comes with a real overfitting risk that has to be actively managed through regularization, early stopping, and a validation-driven hyperparameter search, rather than assumed away.

10 Conclusion

Across this A/B experiment on the Kaggle English Handwritten Characters dataset, the tuned MLP (3 hidden layers, ReLU, Adam, learning rate 0.001, batch size 32, L2 $\alpha = 0.01$, early stopping) substantially outperformed the from-scratch single-layer Perceptron on every metric evaluated – accuracy (51.3% vs 24.3%), macro/weighted precision, recall and F1, and ROC-AUC (0.96 vs 0.84) – confirming that a non-linear, multi-layer representation is necessary to separate 62 visually similar handwritten character classes, while the linear PLA is fundamentally limited by its single decision boundary per class. Systematic hyperparameter tuning (activation, optimizer, learning rate, depth, batch size, regularization) was essential to reach the MLP's final accuracy, improving it by roughly 8 percentage points over an untuned baseline configuration.

11 Learning Outcomes

- Implemented the multiclass perceptron learning algorithm and its weight-update rule from scratch, and observed first-hand its inability to solve a non-linearly-separable, 62-class problem.
- Practiced systematic hyperparameter tuning of a neural network across activation function, optimizer, learning rate, batch size, depth/width, and regularization

strength, and learned to read train-vs-validation accuracy gaps as a diagnostic for overfitting.

- Directly observed the effect of the optimizer (Adam vs SGD) and of L2 regularization / early stopping on training dynamics and generalization.
- Practiced computing and interpreting macro and weighted precision/recall/F1, confusion matrices, and one-vs-rest micro/macro ROC-AUC for a genuinely multi-class (62-class) classification problem.
- Understood, through per-class recall analysis, that a model's hardest classes often line up with humanly-ambiguous look-alike characters, not with arbitrary weaknesses of the classifier.

12 References

1. Kaggle: English Handwritten Characters Dataset – https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset
2. Scikit-learn documentation, `MLPClassifier`: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
3. Rosenblatt, F., "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain", Psychological Review, 1958.
4. Rumelhart, D.E., Hinton, G.E., Williams, R.J., "Learning representations by back-propagating errors", Nature, 1986.